



Universidad Nacional Autónoma de México

Ingeniería en Computación

Compilers

Lexical Analyzer

STUDENTS:

321184995

321331656

321058526

424138220

320560154

Group:

#05

Semester:

2026-II

México, CDMX. 15 March 2025

Contents

1. Introduction	2
2. Theoretical Framework	3
3. Development	3
4. Results	7
5. Conclusions	10

1. Introduction

Compilers are an important tool for computer programming; they are computer programs used to transform high-level source code into low-level target code that the processor can execute directly.

The compiler consists of phases, these are Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation, Code Optimization, and Target Code Generation. For the purposes of the present report, we will focus only on the lexical analysis phase, whose function is to examine the source code of a program and divide it into basic units called tokens. These tokens represent significant elements of the language, such as reserved words, identifiers, operators, constants, and punctuation marks.

However, processing code written in a compiled programming language needs a program that identifies and classifies each element. Without it, this compilation process could not be done.

Hence the need arises for a lexical analyzer that can read a line of code and automatically separate it into lexemes and classify them according to the corresponding token type.

Being the first phase of the compilation process, it carries significant weight as it builds the foundation for the subsequent phases. Its importance lies in the ability to simplify syntactic complexity, optimizing performance by eliminating irrelevant elements, such as whitespace and comments, and serving as the first barrier for error detection.

Understanding its operation is essential as it is a requirement for language design and for our education as computer engineers. By understanding the operation of this phase, we will gain a richer view on how information is processed, allowing us to improve our coding skills, facilitating technical debugging, and understanding the basics of security in data validation

The objective of the project is developing a lexical analyzer capable of processing a text input representing source code and performing an automatic identification of the different elements that compose it, separating the elements of the language and determining whether they correspond to keywords, identifiers, punctuation marks, operators, constants, or character strings. In this way, the program will generate a structured representation of the information found in the code.

Finally, the goal is developing a system that not only correctly identifies each of the tokens present in the input, but also provides clear information about the classification of these elements and the total number of tokens detected, thus allowing a practical understanding of the functioning of lexical analysis within the compilation process.

2. Theoretical Framework

The lexical analysis process is just one phase of a larger process called a compiler, a computer program used to transform high-level source code into low-level target code that the processor can execute directly.

The first phase of the compiler is precisely lexical analysis. The lexical analyzer takes the source code as input, reads its characters and groups them into meaningful units called tokens, along with a terminal symbol that is used for syntax analysis decisions. A token represents a lexical category within a programming language, such as keywords, identifiers, operators, constants, or punctuation marks. An input sequence that forms a token is called a lexeme; therefore, the lexical analyzer isolates the syntax analyzer from the representation of tokens through lexemes.

To perform this recognition, lexical analyzers use pattern identification mechanisms that allow classifying each fragment of the input text. These patterns are usually described using regular expressions, which formally define how certain language elements can be formed.

The main function of the lexical analyzer is to scan the program and group characters into lexemes to later be classified as tokens. These are then sent to the next phase of the compiler, which is the syntax analyzer, to verify that the sequence of tokens complies with the grammatical rules of the language.

The lexical analyzer is also responsible for eliminating whitespace (spaces, tabs, and line breaks) or comments within expressions, so that the syntax analyzer does not have to consider them, avoiding the need to modify the grammar.

We can observe that the lexical analyzer functions as a filter that organizes the program's information before further analysis. [1]

3. Development

The initial CFG proposition is
 $G = (V, \Sigma, P, S)$

$V = \{ \text{Program, Function, StatementList, Statement, Declaration, Type, VarList, Var, Assignment, printf, ReturnLine, Expression, Term, Factor, Value, String} \}$

$\Sigma = \{ \text{int, float, double, main, printf, return, id, int_literal, float_literal, +, -, *, /, =, ,, ;;, (,), , , "text", " } \}$

$P =$

$\text{Program} \rightarrow \text{StatementList} \mid \text{Function}$
 $\text{Function} \rightarrow \text{int main() } \{ \text{StatementList ReturnLine} \}$
 $\text{StatementList} \rightarrow \text{Statement StatementList} \mid \text{Statement}$
 $\text{Statement} \rightarrow \text{Declaration ;} \mid \text{Assignment ;} \mid \text{Printf ;}$
 $\text{Declaration} \rightarrow \text{Type VarList}$
 $\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{double}$
 $\text{VarList} \rightarrow \text{Var} \mid \text{Var , VarList}$
 $\text{Var} \rightarrow \text{id} \mid \text{id = Value}$
 $\text{Assignment} \rightarrow \text{id = Expression}$
 $\text{Printf} \rightarrow \text{printf (String)} \mid \text{printf (String , Expression)}$
 $\text{ReturnLine} \rightarrow \text{return Value ;}$
 $\text{Expression} \rightarrow \text{Expression} + \text{Term} \mid \text{Expression} - \text{Term} \mid \text{Term}$
 $\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Term} / \text{Factor} \mid \text{Factor}$
 $\text{Factor} \rightarrow \text{id} \mid \text{Value}$
 $\text{Value} \rightarrow \text{int_literal} \mid \text{float_literal}$
 $\text{String} \rightarrow \text{string_literal}$

$S = \{\text{Program}\}$

It can form basic texts, but there is left recursion present in some rules. We'll apply this rule to the productions with left recursion

$A \rightarrow \alpha A \mid B$
 Transforms into
 $A \rightarrow BA'$
 $A' \rightarrow \alpha A' \mid \epsilon$

And left factoring with this one

$A \rightarrow \alpha B_1 \mid \alpha B_2$
 Transforms into
 $A \rightarrow \alpha A'$
 $A' \rightarrow B_1 \mid B_2$

The production rules end up as:

$\text{Program} \rightarrow \text{StatementList} \mid \text{Function}$
 $\text{Function} \rightarrow \text{int main() } \{ \text{StatementList ReturnLine} \}$
 $\text{StatementList} \rightarrow \text{Statement StatementList'}$
 $\text{StatementList'} \rightarrow \text{Statement StatementList'} \mid \epsilon$
 $\text{Statement} \rightarrow \text{Declaration ;} \mid \text{Assignment ;} \mid \text{Printf ;}$
 $\text{Declaration} \rightarrow \text{Type VarList}$
 $\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{double}$
 $\text{VarList} \rightarrow \text{Var VarList'}$
 $\text{VarList'} \rightarrow \text{ , Var VarList'} \mid \epsilon$

$\text{Var} \rightarrow \text{id Var}'$
 $\text{Var}' \rightarrow = \text{Value} \mid \epsilon$
 $\text{Assignment} \rightarrow \text{id} = \text{Expression}$
 $\text{Printf} \rightarrow \text{printf} (\text{String Printf}')$
 $\text{Printf}' \rightarrow , \text{Expression} \mid \epsilon$
 $\text{ReturnLine} \rightarrow \text{return Value} ;$
 $\text{Expression} \rightarrow \text{Term Expression}'$
 $\text{Expression}' \rightarrow + \text{Term Expression}' \mid - \text{Term Expression}' \mid \epsilon$
 $\text{Term} \rightarrow \text{Factor Term}'$
 $\text{Term}' \rightarrow * \text{Factor Term}' \mid \text{Factor Term}' \mid \epsilon$
 $\text{Factor} \rightarrow \text{id} \mid \text{Value}$
 $\text{Value} \rightarrow \text{int_literal} \mid \text{float_literal}$
 $\text{String} \rightarrow \text{string_literal}$

For the implementation of the lexical analyzer, the Python programming language was chosen. The development of the program was designed in such a way that the responsibilities of the program could be separated, facilitating the organization of the analysis. The overall application of the program mainly consists of three modules: the main execution module, the module responsible for lexical analysis, and the module that contains the token pattern table.

In the main module, the interaction with the user and the overall control of the program are carried out. This is where a menu is displayed allowing the user to choose between two options: analyzing code from a file or analyzing code entered through the terminal. Once either option is selected, the program verifies that there is user input or, in the case of a file, that the file exists. Once this is confirmed, the program accesses the corresponding content and sends it to the next module responsible for performing the lexical analysis.

Once the user-program interaction is completed, the module responsible for lexical analysis receives the content provided by the user, and subsequently the main token recognition procedure is implemented. This module contains several functions responsible for processing the code and generating the corresponding tokens. Each function is essential to achieve a proper analysis.

The `analyzearchive` function is responsible for performing the analysis when the user provides the code through an external file. This function opens the file specified by the user, reads its entire content, and sends it to the internal procedure that performs the lexical analysis. At the same time, this function handles possible errors related to file reading, such as the nonexistence of the specified path or problems during file opening.

The `analyzeterminal` function performs the analysis when the user enters the code directly through the terminal. In this case, the function receives a text string provided by the user and sends it to the internal procedure that performs the content analysis. Its purpose is to facilitate the quick evaluation of code fragments without the need to use external files.

The main token recognition process is carried out within the `_analizar_contenido` function. This function receives the text to be analyzed and the source from which that content originates. During execution, the program sequentially scans the input text, maintaining control over the current position within the code, as well as the line and column where each element is located. This allows the precise identification of the location of each token within the program.

While scanning the received content, the function compares the sequence of characters with the different patterns defined in the token table. These patterns are represented using regular expressions that allow the recognition of the different lexical categories of the language. When a match is detected between the analyzed text and one of the patterns, the program classifies the corresponding fragment as a token of the indicated type, registers it in an internal list, and continues analyzing the rest of the content. This list is used to store the recognized tokens, where in addition to displaying them along with their classification, the total number of detected tokens is calculated.

If the analyzer encounters a character that does not match any of the defined patterns, the system reports a lexical error indicating the invalid character and the exact position where it was detected. This allows the identification of possible errors in the analyzed source code.

The file that defines the lexical patterns of the analyzer contains the structure that describes the rules used to recognize each type of token within the source code. This component functions as a lexical table, in which the patterns used to identify the different categories of language elements are specified.

The table is composed of pairs formed by a recognition pattern and its associated token type. These patterns are defined using regular expressions, which allow the formal description of the character sequences that correspond to each lexical category. This mechanism enables the analyzer to identify various types of elements present in the code during the analysis process.

The table includes reserved words corresponding to terms that belong to the programming language and that have a specific meaning within the language syntax. Identifiers are also defined, representing names assigned by the programmer to variables, functions, or other elements of the program. Patterns for recognizing constants are also included, which represent literal values within the code. Another important category corresponds to operators, which represent symbols used to perform operations within the language. Additionally, punctuation symbols are included, which delimit structures within the program, such as parentheses, braces, brackets, commas, and semicolons. The table also includes patterns intended to recognize comments and whitespace. However, these elements do not generate tokens during the analysis since they are not part of the syntactic structure that will be processed by later stages of the compiler.

Finally, the module calculates the total number of token types defined within the table, excluding those patterns that correspond to ignored elements such as comments or whitespace. This information makes it possible to determine the variety of lexical categories that the analyzer is capable of recognizing during its execution.

To verify the correct functioning of the lexical analyzer, several tests were performed using different code fragments as input. This allowed us to evaluate the program's ability to identify the various tokens defined in the lexical table.

One of the tests performed consisted of entering a code string that contains a function call for printing and a variable declaration. The following text represents the input used:

```
printf("This is an example");  
int a = 10;
```

During the analysis process, the program identified and classified the different elements according to their lexical category. The detected tokens included identifiers, string constants, assignment operators, punctuation symbols, and reserved words.

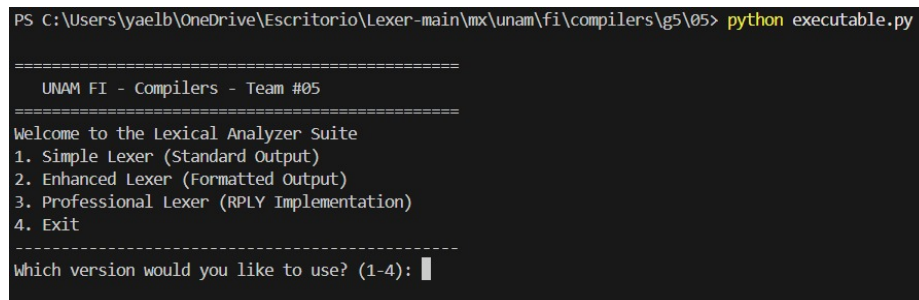
As a result, the program produced a list of tokens representing the sequence of lexical elements found in the input. Additionally, the system reported the total number of identified tokens, allowing verification that all components of the code were correctly recognized by the analyzer.

Additionally, tests were performed by entering the code directly through the program's terminal. In this case, the system allowed multiple lines of code to be entered until the user indicated the end of the input. Afterwards, the analyzer processed the entered content and displayed the detected tokens along with their corresponding classification.

The tests performed confirm that the lexical analyzer was able to process both external files and inputs manually entered by the user, correctly identifying the different types of tokens defined within the system.

4. Results

This section will present the results obtained from testing the program. This time, you have the option to implement up to 3 versions Simple Lexer (Standard Output), Enhanced Lexer (Formatted Output) and Professional Lexer (RPLY Implementation).



```
PS C:\Users\yaelb\OneDrive\Escritorio\Lexer-main\mx\unam\fi\compilers\g5\05> python executable.py  
  
=====   
UNAM FI - Compilers - Team #05  
=====   
Welcome to the Lexical Analyzer Suite  
1. Simple Lexer (Standard Output)  
2. Enhanced Lexer (Formatted Output)  
3. Professional Lexer (RPLY Implementation)  
4. Exit  
-----   
Which version would you like to use? (1-4): █
```

Figure 1

The figure above represents the start of the lexical analyzer's execution. As you can see, three options appear, as mentioned before.

We will begin with the first option in each case.

```
Which version would you like to use? (1-4): 1

--- Running: src ---
Select how you will enter your code (archive/terminal): archive
Enter your file path:
hola.txt
keyword punctuation literal punctuation
Total of tokens: 5

=====
UNAM FI - Compilers - Team #05
=====
Welcome to the Lexical Analyzer Suite
1. Simple Lexer (Standard Output)
2. Enhanced Lexer (Formatted Output)
3. Professional Lexer (RPLY Implementation)
4. Exit
-----
Which version would you like to use? (1-4):
```

(a)

```
≡ hola.txt ×
mx > unam > fi > compilers > g5 > 05 > ≡ hola.txt
1 print("hola mundo");
```

(b)

Figure 2

The operation of Simple Lexer is shown using a file as input (a). The contents of the file hola.txt are also shown (b).

Now, using the same Simple Lexer, we will use the terminal as input.

```
=====
UNAM FI - Compilers - Team #05
=====
Welcome to the Lexical Analyzer Suite
1. Simple Lexer (Standard Output)
2. Enhanced Lexer (Formatted Output)
3. Professional Lexer (RPLY Implementation)
4. Exit
-----
Which version would you like to use? (1-4): 1

--- Running: src ---
Select how you will enter your code (archive/terminal): terminal
Enter your code:
int x = 45;
keyword identifier operator constant punctuation
Total of tokens: 5

=====
UNAM FI - Compilers - Team #05
=====
Welcome to the Lexical Analyzer Suite
1. Simple Lexer (Standard Output)
2. Enhanced Lexer (Formatted Output)
3. Professional Lexer (RPLY Implementation)
4. Exit
-----
Which version would you like to use? (1-4):
```

Figure 3

As you can see in the image, we add the following entry: `int x = 45;` And we obtain the lexical analysis as output.

Now we will use the Enhanced Lexer version and follow the same protocol as before (first we'll try with a file and then with the terminal).

```

Which version would you like to use? (1-4): 2

--- Running: src_recharged ---
=====
Welcome to Lexer from Team 5
UNAM FI Compilers - Group 5
=====

Options:
1. Read from file (archive)
2. Read from terminal
3. Exit

Select how you will enter your code (archive/terminal/exit): 1
Enter your file path:
hola.txt

Analyzing file: hola.txt
-----

Input code:
print("hola mundo");

Tokens found:
-----
keyword      | 'print' | line 1, col 1
punctuation  | '('     | line 1, col 6
constant     | "hola mundo" | line 1, col 7
punctuation  | ')'     | line 1, col 19
punctuation  | ';'     | line 1, col 20
-----

Total de tokens encontrados: 5
Fuente: FILE

Output format (required):
keyword punctuation constant punctuation punctuation

```

(a)

```

Options:
1. Read from file (archive)
2. Read from terminal
3. Exit

Select how you will enter your code (archive/terminal/exit): 2
Enter your code (end with Ctrl+D on empty line or type 'END' on new line):
printf("HELLO WORLD"); int x = 3;
END

Analyzing terminal input:
-----

Input code:
printf("HELLO WORLD"); int x = 3;

Tokens found:
-----
Identifier   | 'printf' | line 1, col 1
punctuation  | '('     | line 1, col 7
constant     | "HELLO WORLD" | line 1, col 8
punctuation  | ')'     | line 1, col 22
punctuation  | ';'     | line 1, col 23
keyword      | 'int'    | line 1, col 25
Identifier   | 'x'      | line 1, col 29
operator      | '='      | line 1, col 31
constant     | '3'      | line 1, col 33
punctuation  | ';'      | line 1, col 34
-----

Total de tokens encontrados: 10
Fuente: TERMINAL

Output format (required):
Identifier punctuation constant punctuation punctuation keyword identifier operator constant punctuation

```

(b)

Figure 4

Image (a) shows a file used as input (Figure 2.b) for the lexical analyzer version two. Similarly, image (b) shows the input from the terminal for the lexical analysis process. The output displays the token count and its classification.

Once the results for version 2 are finalized, we proceed to show the operation with version 3 Professional Lexer, both with a file and with the terminal.

```

=====
UNAM FI - Compilers - Team #05
=====
Welcome to the Lexical Analyzer Suite
1. Simple Lexer (Standard Output)
2. Enhanced Lexer (Formatted Output)
3. Professional Lexer (RPLY Implementation)
4. Exit
-----
Which version would you like to use? (1-4): 3

--- Running: src_rply ---
Select how you will enter your code (archive/terminal): archive
Enter your file path:
hola.txt
keyword punctuation constant punctuation punctuation
Total of tokens: 5

```

(a)

```

=====
UNAM FI - Compilers - Team #05
=====
Welcome to the Lexical Analyzer Suite
1. Simple Lexer (Standard Output)
2. Enhanced Lexer (Formatted Output)
3. Professional Lexer (RPLY Implementation)
4. Exit
-----
Which version would you like to use? (1-4): 3

--- Running: src_rply ---
Select how you will enter your code (archive/terminal): terminal
Enter your code:
String s; int x = 0; printf("HELLO WORLD");
Identifier Identifier punctuation keyword Identifier operator constant punctuation punctuation
Total of tokens: 13
-----

=====
UNAM FI - Compilers - Team #05
=====
Welcome to the Lexical Analyzer Suite
1. Simple Lexer (Standard Output)
2. Enhanced Lexer (Formatted Output)
3. Professional Lexer (RPLY Implementation)
4. Exit
-----
Which version would you like to use? (1-4): 1

```

(b)

Figure 5

Image (a) shows a file used as input (Figure 2.b) for the lexical analyzer version three. Similarly, image (b) shows the input from the terminal for the lexical analysis process. The output displays the token count and its classification.

5. Conclusions

The development of the lexical analyzer facilitated the application of the theoretical foundations of the lexical analysis phase in compiler construction, demonstrating the feasibility of implementing a token recognizer based on regular expressions and finite automata. Through the separation of the program into independent modules, user interface, lexical processing, and pattern table, a clean and maintainable architecture was achieved that reflects good software engineering practices.

The implementation proved that regular expressions are adequate for modeling the lexical patterns of a simplified programming language, correctly classifying keywords, identifiers, operators, constants, and punctuation marks. Likewise, lexical error handling through the notification of invalid characters with their exact location (line and column) proved to be a valuable feature for source code debugging.

The design of the analyzer as a reusable component, capable of processing inputs from both files and the terminal, demonstrates the flexibility necessary to be integrated into later stages of the compilation process. The modular architecture adopted lays the groundwork for the future incorporation of the syntactic and semantic analysis phases, maintaining a clear separation of responsibilities.

In conclusion, the project fulfilled the objective of building a functional tool that not only identifies and classifies the lexical elements of a program but also provides structured information about the tokens found, demonstrating the practical applicability of the theoretical concepts studied in the course.

Referencias

- [1] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2.^a ed. Boston: Addison-Wesley, 2006.